
RAPPORT DE PROJET

TRADE BLOTTER

Conception et développement d'une plateforme
de gestion et de contrôle post-trade

Trade Capture • Confirmations • Matching
Positions • Audit & Controls

Auteur : Gaspar RAYBAUD-MABILY

Technologies : Python, Streamlit, SQLite, Plotly, ReportLab

Nature : Projet personnel / Portfolio

Année : 2026

Post-Trade Management & Control Platform

Application de démonstration utilisant exclusivement des données synthétiques

Avertissement

Le projet présenté dans ce rapport est une application personnelle de démonstration développée dans un objectif d'apprentissage et de portfolio.

Toutes les transactions, contreparties, utilisateurs, identifiants, montants et données affichés sont fictifs ou synthétiques.

L'application ne constitue pas un système de production et n'est pas destinée à traiter des transactions financières réelles.

Les documents PDF produits par l'application sont des récapitulatifs de démonstration et ne constituent pas des confirmations juridiques de transactions.

Résumé

Le projet **Trade Blotter** consiste en la conception et le développement d'une application web simulant un environnement de gestion et de contrôle post-trade.

L'objectif n'était pas uniquement de construire une interface permettant d'afficher une liste de transactions. Le projet cherche à représenter un cycle de vie cohérent : une transaction est saisie, validée par des règles métier, enregistrée dans une base relationnelle, suivie dans un blotter, puis intégrée à un workflow de confirmation. Les opérations réalisées sont également enregistrées dans une piste d'audit.

Trois familles de produits financiers sont représentées : les obligations (*Bonds*), les swaps de taux d'intérêt (*Interest Rate Swaps – IRS*) et les contrats de change à terme (*FX Forwards*).

L'application comprend également un *Post-Trade Control Room* permettant de suivre l'état du workflow, les exceptions opérationnelles, l'activité de trading et différentes vues de positions nominales.

Sur le plan technique, le projet repose principalement sur Python, Streamlit, SQLite, Pandas, Plotly, ReportLab et Pytest. Le code est organisé de manière à distinguer l'interface utilisateur, les services métier et la couche de persistance.

Enfin, l'application a été versionnée avec Git, publiée sur GitHub puis déployée sur Streamlit Community Cloud afin d'obtenir une démonstration publique et directement accessible.

Table des matières

Avertissement	1
Résumé	2
I Contexte et conception du projet	10
1 Introduction	11
1.1 Origine du projet	11
1.2 Problématique	11
1.3 Objectifs	12
1.4 Périmètre du projet	12
2 Compréhension du processus post-trade	13
2.1 De l'exécution au contrôle	13
2.2 Cycle de vie simplifié	13
2.3 Importance du temps	14
3 Architecture et choix technologiques	15
3.1 Principes d'architecture	15
3.2 Organisation du dépôt	16
3.3 Pourquoi Python ?	16
3.4 Pourquoi Streamlit ?	17
3.5 Pourquoi SQLite ?	17
3.6 Technologies complémentaires	18
4 Modélisation des données	19
4.1 Problème de modélisation	19
4.2 Table commune des transactions	19

4.3	Tables spécifiques	20
4.4	Contreparties et audit	20
II	Conception fonctionnelle	21
5	Modélisation des produits financiers	22
5.1	Principe général	22
5.2	Bond	22
5.3	Interest Rate Swap	22
5.4	FX Forward	23
5.5	Pourquoi “Position Overview” ?	23
6	Trade Capture et validation métier	24
6.1	Objectif du module	24
6.2	Une distinction fondamentale : UX et intégrité	24
6.3	Flux de validation	25
6.4	Règles communes	25
6.5	Validation Bond	26
6.6	Validation IRS	26
6.7	Validation FX Forward	26
6.8	Présentation des erreurs	26
7	Trade Blotter et consultation	28
7.1	Rôle du blotter	28
7.2	Lisibilité des données	28
7.3	Économie du trade	29
7.4	Vue détaillée	29
8	Post-Trade Control Room	30
8.1	Objectif	30
8.2	Indicateurs de workflow	30
8.3	Exception management	31
8.4	Oldest Open Exceptions	31
8.5	Position Overview	31
8.6	Trading Analytics	32
9	Workflow de confirmation	33

9.1	De la transaction au Trade Recap	33
9.2	Étape BOOKED	33
9.3	Étape GENERATED	33
9.4	Étape SENT	33
9.5	Étape MATCHED	34
9.6	Interface dynamique	34
10	Génération du Trade Recap PDF	35
10.1	Objectif	35
10.2	Organisation du document	36
10.3	Snapshot documentaire	37
10.4	Restauration du document	37
11	Rôles et permissions	38
11.1	Pourquoi introduire des rôles ?	38
11.2	Trader	38
11.3	Operations	38
11.4	Supervisor	38
11.5	Interface et backend	38
12	Audit Trail	39
12.1	Objectif de la traçabilité	39
12.2	Source des timestamps	39
III	Réalisation technique	41
13	Services métier	42
13.1	Trade Service	42
13.2	Confirmation Service	42
13.3	Audit Service	42
13.4	Counterparty Service	42
13.5	User Service	42
13.6	Position Service	43
13.7	Email Service	43
14	Dataset synthétique	44
14.1	Pourquoi générer des données ?	44

14.2 Cohérence du dataset	44
14.3 Données obligatoires	44
15 Tests et qualité	46
15.1 Pourquoi tester le backend ?	46
15.2 Scénarios testés	46
15.3 Principe d'intégrité	46
16 Interface et expérience utilisateur	47
16.1 Direction visuelle	47
16.2 Navigation	47
16.3 Page d'accueil	47
IV Du développement au déploiement	48
17 Versionnement avec Git	49
17.1 Objectif	49
17.2 Fichiers exclus	49
17.3 Dépôt public	49
18 Déploiement sur Streamlit Community Cloud	50
18.1 Objectif du déploiement	50
18.2 Problème de la base vide	50
18.3 Solution retenue	50
18.4 Pourquoi ne pas appeler seed à chaque démarrage ?	51
18.5 Persistance et limite du cloud	51
V Analyse critique et perspectives	52
19 Résultats obtenus	53
19.1 Résultat fonctionnel	53
19.2 Résultat technique	53
20 Limites	54
20.1 Authentification	54
20.2 Base de données	54
20.3 Confirmations	54

20.4 Risk Management	54
20.5 Market data	55
21 Perspectives d'évolution	56
21.1 Module Risk Management	56
21.2 API et séparation frontend/backend	56
21.3 Matching automatisé	57
21.4 Gestion documentaire	57
22 Conclusion	58

Table des figures

3.1	Architecture logique de Trade Blotter	15
6.1	Écran Book Trade	24
6.2	Principe de validation d'une transaction	25
7.1	Vue principale du Trade Blotter	28
7.2	Détail d'un FX Forward	29
8.1	Post-Trade Control Room	30
8.2	Position Overview par type de produit	31
8.3	Trade Activity Over Time	32
9.1	Interface de gestion des confirmations	34
10.1	Exemple de Trade Recap	36
12.1	Piste d'audit de l'application	39

Liste des tableaux

3.1	Stack technique	18
8.1	Mesures utilisées dans Position Overview	32

Première partie

Contexte et conception du projet

Chapitre 1

Introduction

1.1 Origine du projet

Le Trade Blotter a été développé avec l'objectif de construire une application combinant deux dimensions : une dimension financière et opérationnelle d'une part, et une dimension logicielle d'autre part.

Une transaction financière possède une structure économique, un cycle de vie, des statuts, des contrôles et des événements associés.

L'objectif du projet était donc de dépasser la création d'une simple interface CRUD (*Create, Read, Update, Delete*) pour construire un petit système cohérent autour de la transaction.

Le choix du post-trade est particulièrement intéressant pour cet objectif, car il permet d'introduire plusieurs problématiques complémentaires : modélisation de produits, validation métier, workflow, contrôle opérationnel, génération documentaire, audit et gestion des rôles.

1.2 Problématique

La problématique générale peut être formulée ainsi :

Comment construire une application légère mais cohérente permettant de représenter plusieurs produits financiers, de contrôler leur saisie, de suivre leur traitement post-trade et de fournir une vision opérationnelle de l'ensemble du portefeuille de démonstration ?

Cette question implique plusieurs sous-problèmes.

Comment représenter dans une même application des produits dont les caractéristiques économiques sont différentes ?

Comment empêcher l'enregistrement d'une transaction incohérente ?

Comment représenter l'avancement d'une confirmation ?

Comment identifier rapidement les opérations nécessitant une attention opérationnelle ?

Comment conserver la trace des actions effectuées ?

Enfin, comment construire l'ensemble sans introduire une infrastructure disproportionnée par rapport à un projet de portfolio ?

1.3 Objectifs

Les objectifs retenus sont les suivants :

- construire un Trade Blotter multi-produits ;
- représenter des Bonds, IRS et FX Forwards ;
- implémenter des règles de validation métier ;
- construire un workflow de confirmation ;
- permettre le suivi des exceptions opérationnelles ;
- proposer des indicateurs d'activité et de positions ;
- différencier plusieurs rôles utilisateurs ;
- générer des récapitulatifs PDF ;
- conserver une piste d'audit ;
- tester les règles importantes côté backend ;
- produire un dataset synthétique cohérent ;
- rendre l'application publiquement accessible.

1.4 Périmètre du projet

Le projet ne cherche pas à reproduire l'ensemble d'une infrastructure de marché.

Il ne comporte notamment pas de moteur complet de valorisation, de P&L, de calcul réglementaire, de connexion à une plateforme de marché ou de gestion réelle du règlement-livraison.

Ce périmètre est volontaire. L'objectif est de construire une démonstration cohérente du workflow et des contrôles post-trade tout en conservant une architecture suffisamment lisible pour être comprise et présentée.

Chapitre 2

Compréhension du processus post-trade

2.1 De l'exécution au contrôle

Après l'exécution économique d'une transaction, plusieurs opérations restent nécessaires.

Les caractéristiques de la transaction doivent être correctement enregistrées. Les deux parties doivent disposer d'une représentation cohérente de l'opération. Les équipes opérationnelles doivent pouvoir suivre son état et identifier les éventuelles anomalies ou retards.

Dans Trade Blotter, ce processus est représenté de manière simplifiée par plusieurs composants qui communiquent entre eux :

- **Book Trade** pour la saisie ;
- **Trade Blotter** pour la consultation ;
- **Confirmations** pour le workflow post-trade ;
- **Dashboard** pour le contrôle ;
- **Audit Trail** pour la traçabilité.

Ces écrans ne sont donc pas conçus comme des fonctionnalités indépendantes. Ils représentent différentes vues du même cycle de vie.

2.2 Cycle de vie simplifié

Le workflow principal de confirmation est :

BOOKED → GENERATED → SENT → MATCHED

Une transaction **BOOKED** existe dans le système mais ne possède pas encore de confirmation générée.

L'état **GENERATED** signifie qu'un Trade Recap a été produit à partir des caractéristiques économiques enregistrées.

L'état **SENT** représente l'envoi de la confirmation. Dans le cadre du projet, cet envoi est simulé et aucun message réel n'est adressé à une contrepartie.

Enfin, **MATCHED** indique que la transaction est considérée comme rapprochée dans le workflow de démonstration.

2.3 Importance du temps

Le statut seul n'est pas suffisant pour analyser une transaction.

Une transaction **SENT** depuis quelques secondes n'a pas la même signification opérationnelle qu'une transaction **SENT** restant non matchée depuis plusieurs jours.

Le projet associe donc le statut à des timestamps et utilise l'âge des transactions pour construire certains indicateurs d'exception.

Cette dimension temporelle permet de transformer une simple liste de statuts en outil de contrôle.

Chapitre 3

Architecture et choix technologiques

3.1 Principes d'architecture

Une première possibilité aurait consisté à placer toute la logique dans `app.py`. Cette solution aurait été rapide au début du développement, mais elle aurait progressivement mélangé :

- l’affichage Streamlit ;
- les règles métier ;
- les requêtes SQL ;
- la génération des confirmations ;
- la gestion de l’audit.

Le projet adopte donc une séparation entre plusieurs responsabilités.

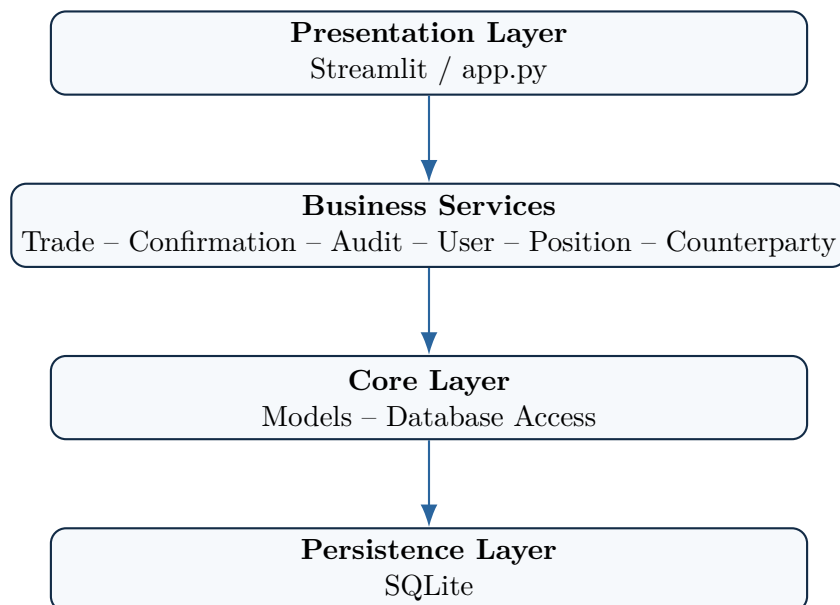


Figure 3.1 – *Architecture logique de Trade Blotter*

L’interface est responsable de la présentation et de l’interaction avec l’utilisateur.

Les services contiennent la logique métier.

Le module `core` regroupe les structures centrales et l’accès à la base.

SQLite assure finalement la persistance.

3.2 Organisation du dépôt

```
trade-blotter/  
|  
|-- app.py  
|-- seed_demo.py  
|-- requirements.txt  
|-- README.md  
|  
|-- core/  
|   |-- database.py  
|   |-- models.py  
|  
|-- services/  
|   |-- audit_service.py  
|   |-- confirmation_service.py  
|   |-- counterparty_service.py  
|   |-- email_service.py  
|   |-- exposure_service.py  
|   |-- trade_service.py  
|   |-- user_service.py  
|  
|-- data/  
|   |-- generated/  
|  
|-- docs/  
|   |-- screenshots/  
|  
|-- tests/
```

Cette organisation rend plus visible la responsabilité de chaque composant et évite que l'interface Streamlit devienne l'unique endroit contenant toute la logique de l'application.

3.3 Pourquoi Python ?

Python a été retenu car il permet de traiter dans un même environnement la logique métier, la manipulation des données, la génération de documents, les tests et l'interface Streamlit.

Ce choix permet également de conserver une base de code homogène.

Pour un projet de cette taille, l'utilisation d'un langage unique réduit la complexité technique et permet de concentrer davantage d'efforts sur le fonctionnement du système.

3.4 Pourquoi Streamlit ?

Streamlit permet de construire rapidement une interface interactive à partir de Python.

Pour Trade Blotter, cette caractéristique est particulièrement adaptée : les tableaux, filtres, formulaires, graphiques et boutons de workflow peuvent être reliés directement aux services Python.

L'alternative aurait été de construire séparément un backend et un frontend, par exemple avec une API Python et une interface JavaScript.

Cette architecture aurait été plus proche de certaines applications de production, mais aurait également augmenté fortement la taille du projet.

Streamlit représente donc ici un compromis entre rapidité de développement, interactivité et lisibilité.

Ce choix est adapté à une démonstration et ne signifie pas que Streamlit serait automatiquement le choix retenu pour une plateforme post-trade institutionnelle multi-utilisateurs.

3.5 Pourquoi SQLite ?

Le modèle de données du projet est naturellement relationnel.

Une transaction possède des données communes mais peut également être liée à des informations spécifiques à son produit, à une contrepartie et à plusieurs événements d'audit.

SQLite permet de représenter ces relations sans nécessiter le déploiement d'un serveur de base de données séparé.

Pour une application autonome et un projet de portfolio, cette simplicité est un avantage important.

Dans une architecture de production multi-utilisateurs, une base serveur telle que PostgreSQL serait cependant plus appropriée pour la concurrence, la gestion des connexions, la résilience et l'exploitation.

3.6 Technologies complémentaires

Technologie	Rôle dans le projet
Python	Logique générale et services métier
Streamlit	Interface utilisateur
SQLite	Persistance relationnelle
Pandas	Manipulation et agrégation des données
Plotly	Graphiques interactifs
ReportLab	Génération des Trade Recap PDF
Pytest	Tests automatisés
Git	Gestion des versions
GitHub	Hébergement public du code source
Streamlit Community Cloud	Hébergement de la démonstration

Table 3.1 – *Stack technique*

Chapitre 4

Modélisation des données

4.1 Problème de modélisation

Une difficulté du projet vient du fait que les trois produits ne partagent pas exactement les mêmes caractéristiques.

Un Bond possède un ISIN, un prix et un nominal.

Un IRS possède un notionnel, un taux fixe et un indice flottant.

Un FX Forward possède deux devises, deux montants et un taux forward.

Placer tous ces champs dans une seule table aurait conduit à de nombreuses colonnes inutilisées selon le produit.

La solution retenue consiste à séparer les informations communes des informations spécifiques.

4.2 Table commune des transactions

La table centrale **trades** contient les informations applicables à l'ensemble des produits, telles que :

- Trade ID ;
- Trade Date ;
- heure de transaction ;
- trader ;
- desk ;
- produit ;
- contrepartie ;
- broker ;
- statut de workflow ;
- statut de confirmation ;
- commentaires ;
- date de création.

4.3 Tables spécifiques

Les caractéristiques propres à chaque produit sont stockées séparément :

- `bond_details`;
- `irs_details`;
- `fx_forward_details`.

Cette approche permet de conserver un blotter commun tout en respectant les différences économiques entre les produits.

4.4 Contreparties et audit

La table `counterparties` centralise les contreparties synthétiques utilisées dans l'application.

La table `audit_events` conserve les événements du cycle de vie.

L'audit n'est donc pas reconstruit uniquement à partir du statut courant : il conserve également les actions qui ont conduit à cet état.

Deuxième partie

Conception fonctionnelle

Chapitre 5

Modélisation des produits financiers

5.1 Principe général

Le Trade Blotter utilise une structure commune pour permettre la consultation de toutes les transactions, mais conserve une représentation économique spécifique à chaque produit.

Cette distinction est importante : une application multi-produits ne peut pas simplement supposer que toutes les transactions sont caractérisées par un montant et un prix.

5.2 Bond

Une transaction obligataire comprend notamment :

- ISIN ;
- description ;
- BUY / SELL ;
- Face Amount ;
- Clean Price ;
- devise ;
- Settlement Date ;
- Maturity Date.

Le nominal est signé selon le sens de la transaction :

$$P_{\text{Bond}} = \begin{cases} +N & \text{si BUY} \\ -N & \text{si SELL} \end{cases}$$

où N représente le Face Amount.

Cette représentation permet d'obtenir une *Net Face Position* par devise.

Elle ne représente cependant ni la valeur de marché du portefeuille, ni son risque de taux.

5.3 Interest Rate Swap

Pour un IRS, les informations principales sont :

- Notional ;
- Currency ;
- PAY FIXED / RECEIVE FIXED ;
- Fixed Rate ;
- Floating Index ;
- Effective Date ;
- Maturity Date.

La convention retenue dans le dashboard est :

$$P_{\text{IRS}} = \sum N_{\text{Receive Fixed}} - \sum N_{\text{Pay Fixed}}$$

Ainsi, RECEIVE FIXED contribue positivement à la vue et PAY FIXED négativement.

Le résultat est nommé *Net Fixed-Leg Notional*.

Cette convention est utile pour résumer l'orientation nominale des positions, mais elle ne doit pas être assimilée à une DV01 ou à une mesure complète du risque de taux.

5.4 FX Forward

Un FX Forward implique simultanément deux jambes.

Exemple :

+10 000 000 EUR

−11 300 000 USD

avec un taux forward de :

1.1300 USD/EUR

Une représentation mono-devise serait donc insuffisante.

Le dashboard agrège séparément les montants achetés et vendus afin de construire une *Net Currency Position*.

5.5 Pourquoi “Position Overview” ?

Une première terminologie possible aurait été d'utiliser le terme *Exposure*.

Cependant, les grandeurs calculées par l'application sont essentiellement des positions nominales.

Or une exposition au risque au sens financier peut nécessiter des éléments supplémentaires : prix de marché, courbes de taux, volatilités, duration, sensibilités ou scénarios.

La section a donc été nommée **Position Overview**, terminologie plus précise au regard des calculs réellement effectués.

Chapitre 6

Trade Capture et validation métier

6.1 Objectif du module

Le module Book Trade permet à un utilisateur ayant le rôle Trader de créer une nouvelle transaction.

L'interface change dynamiquement selon le produit sélectionné afin de ne présenter que les champs économiquement pertinents.

The screenshot displays the 'Book Trade' interface. On the left is a sidebar with a 'TRADE BLOTTER' header and a 'Current User' dropdown set to 'Alex Martin - Trader'. Below this are navigation buttons: 'Home', 'Dashboard', 'Trade Blotter', 'Book Trade' (highlighted), 'Confirmations', and 'Audit Trail'. At the bottom of the sidebar, it indicates 'DEMO ENVIRONMENT' and '100% synthetic data'. The main area is titled 'Book Trade' with a subtitle 'Capture a new transaction using product-specific economics.' It contains several input fields: 'Product' (Bond), 'Trade date' (2026/09/25), 'Trader' (Alex Martin), 'Counterparty' (Atlas Bank Demo), 'Broker' (Demo Broker), and 'Comments' (Manual demo trade). Below these is the 'Bond economics' section with fields for 'Description' (OAT Demo 3.00% 2031), 'ISIN' (FR00000DEMO1), 'Side' (BUY), 'Face amount' (5000000.00), 'Clean price' (99.42), 'Currency' (EUR), 'Maturity date' (2031/09/24), and 'Settlement date' (2026/09/27). A 'Book Trade' button is at the bottom.

Figure 6.1 – Écran Book Trade

6.2 Une distinction fondamentale : UX et intégrité

Lors du développement, une distinction importante a été retenue entre les contraintes de l'interface et la validation métier.

Un widget peut limiter visuellement certaines valeurs. Cependant, cette restriction ne constitue pas une garantie suffisante de l'intégrité des données.

La règle adoptée est donc :

Les contraintes des widgets améliorent l'expérience utilisateur. La validation backend constitue l'autorité pour décider si une transaction peut être enregistrée.

Cette décision évite que l'intégrité de la base dépende uniquement du comportement d'un composant graphique.

6.3 Flux de validation

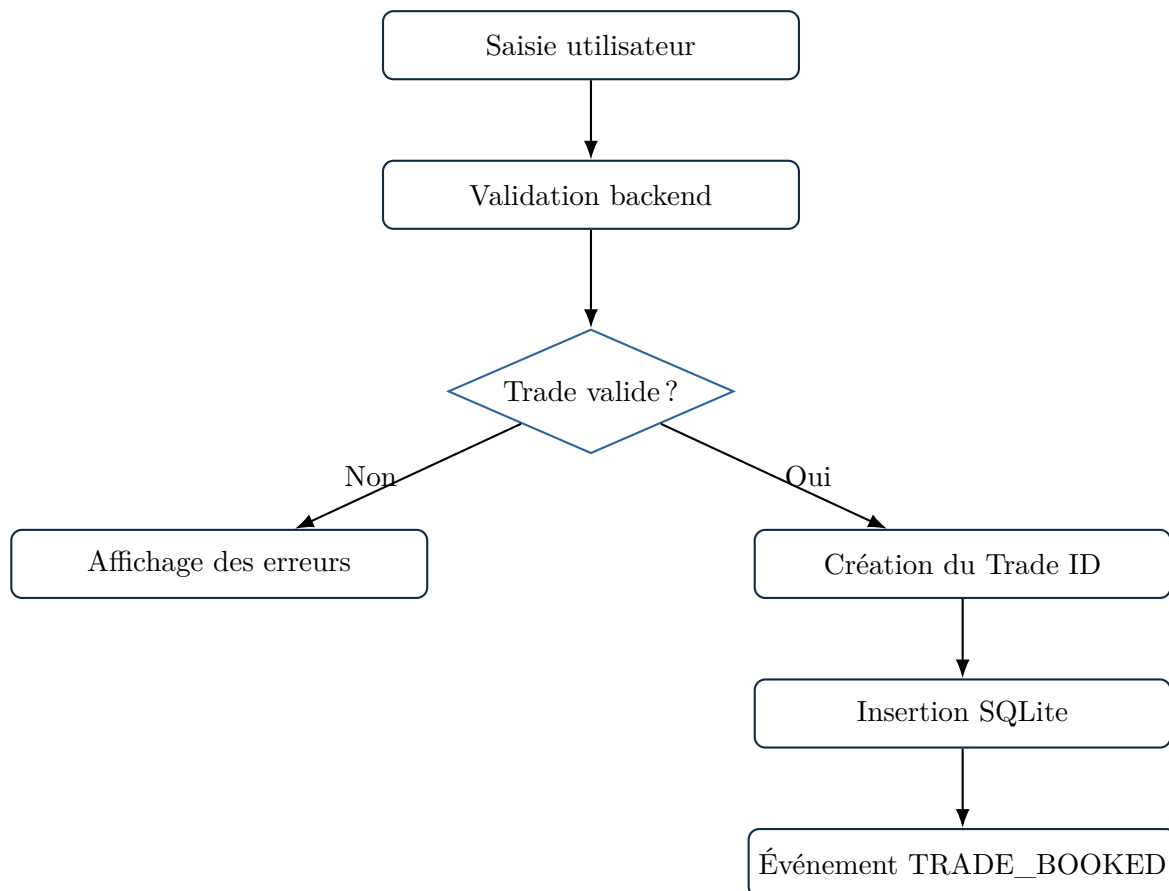


Figure 6.2 – *Principe de validation d'une transaction*

Une transaction invalide ne doit donc jamais recevoir de Trade ID ni produire un événement TRADE_BOOKED.

6.4 Règles communes

Plusieurs contrôles sont indépendants du produit.

La Trade Date ne peut notamment pas être située dans le futur.

Les champs communs nécessaires à l'identification et au traitement de la transaction doivent également être renseignés.

Les erreurs sont cumulées afin que l'utilisateur puisse corriger plusieurs problèmes lors d'une

seule itération.

6.5 Validation Bond

Les règles comprennent notamment :

- ISIN de 12 caractères ;
- deux premiers caractères alphabétiques ;
- Face Amount strictement positif ;
- Clean Price strictement positif ;
- Settlement Date supérieure ou égale à Trade Date ;
- Maturity Date postérieure à Settlement Date.

Le contrôle de l'ISIN est volontairement structurel et ne cherche pas à implémenter une validation complète d'un véritable identifiant de marché.

6.6 Validation IRS

Pour un IRS :

- le Notional doit être positif ;
- le Fixed Rate ne peut pas être négatif ;
- Effective Date doit être supérieure ou égale à Trade Date ;
- Maturity Date doit être postérieure à Effective Date.

6.7 Validation FX Forward

Pour un FX Forward :

- Buy Amount doit être positif ;
- Sell Amount doit être positif ;
- Forward Rate doit être positif ;
- les devises achetée et vendue doivent être différentes ;
- les devises doivent être cohérentes avec la paire ;
- Value Date doit être postérieure à Trade Date.

6.8 Présentation des erreurs

L'interface associe les erreurs aux champs concernés afin d'éviter un simple message générique difficile à interpréter.

Après une tentative de booking invalide, un récapitulatif global permet également de visualiser l'ensemble des corrections nécessaires.

Cette combinaison fournit deux niveaux d'information :

- une information locale au niveau du champ ;
- une information globale au niveau de la transaction.

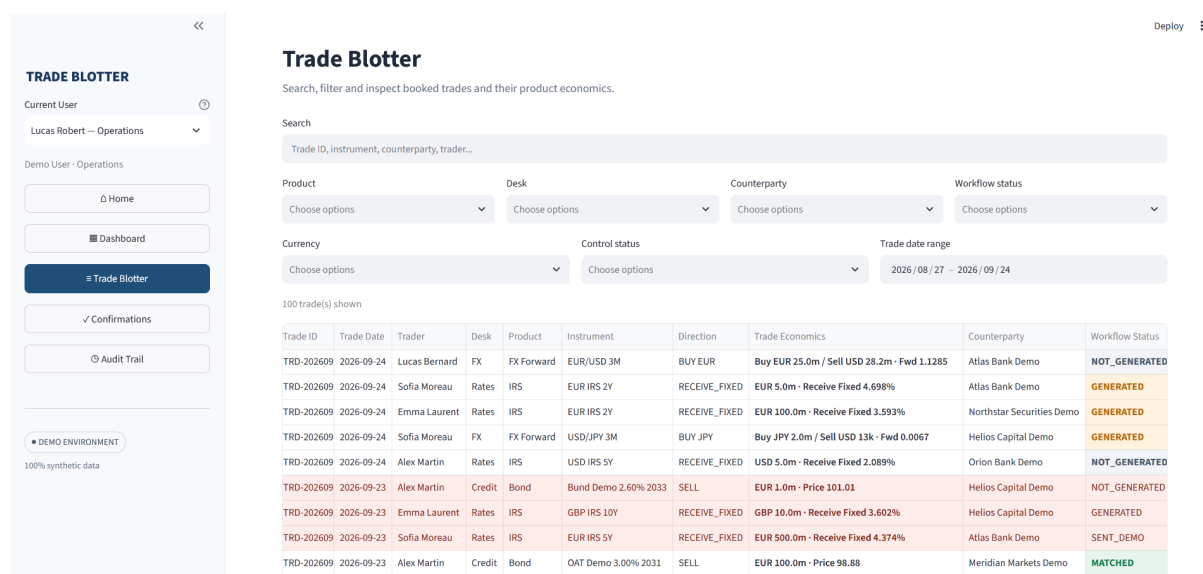
Chapitre 7

Trade Blotter et consultation

7.1 Rôle du blotter

Le Trade Blotter constitue la vue centrale des transactions.

Son objectif est de permettre à l'utilisateur d'obtenir rapidement une vue d'ensemble tout en conservant la possibilité d'accéder au détail économique d'une opération.



TRADE BLOTTER

Current User: Lucas Robert — Operations

Demo User: Operations

Home Dashboard Trade Blotter Confirmations Audit Trail

DEMO ENVIRONMENT
100% synthetic data

Trade Blotter
Search, filter and inspect booked trades and their product economics.

Search: Trade ID, instrument, counterparty, trader...

Product: Choose options Desk: Choose options Counterparty: Choose options Workflow status: Choose options

Currency: Choose options Control status: Choose options Trade date range: 2026 / 08 / 27 - 2026 / 09 / 24

100 trade(s) shown

Trade ID	Trade Date	Trader	Desk	Product	Instrument	Direction	Trade Economics	Counterparty	Workflow Status
TRD-202609	2026-09-24	Lucas Bernard	FX	FX Forward	EUR/USD 3M	BUY EUR	Buy EUR 25.0m / Sell USD 28.2m · Fwd 1.1285	Atlas Bank Demo	NOT_GENERATED
TRD-202609	2026-09-24	Sofia Moreau	Rates	IRS	EUR IRS 2Y	RECEIVE_FIXED	EUR 5.0m · Receive Fixed 4.698%	Atlas Bank Demo	GENERATED
TRD-202609	2026-09-24	Emma Laurent	Rates	IRS	EUR IRS 2Y	RECEIVE_FIXED	EUR 100.0m · Receive Fixed 3.593%	Northstar Securities Demo	GENERATED
TRD-202609	2026-09-24	Sofia Moreau	FX	FX Forward	USD/JPY 3M	BUY JPY	Buy JPY 2.0m / Sell USD 13k · Fwd 0.0067	Helios Capital Demo	GENERATED
TRD-202609	2026-09-24	Alex Martin	Rates	IRS	USD IRS 5Y	RECEIVE_FIXED	USD 5.0m · Receive Fixed 2.089%	Orion Bank Demo	NOT_GENERATED
TRD-202609	2026-09-23	Alex Martin	Credit	Bond	Bund Demo 2.60% 2033	SELL	EUR 1.0m · Price 101.01	Helios Capital Demo	NOT_GENERATED
TRD-202609	2026-09-23	Emma Laurent	Rates	IRS	GBP IRS 10Y	RECEIVE_FIXED	GBP 10.0m · Receive Fixed 3.602%	Helios Capital Demo	GENERATED
TRD-202609	2026-09-23	Sofia Moreau	Rates	IRS	EUR IRS 5Y	RECEIVE_FIXED	EUR 500.0m · Receive Fixed 4.374%	Atlas Bank Demo	SENT_DEMO
TRD-202609	2026-09-23	Alex Martin	Credit	Bond	OAT Demo 3.00% 2031	SELL	EUR 100.0m · Price 98.88	Meridian Markets Demo	MATCHED

Figure 7.1 — Vue principale du Trade Blotter

7.2 Lisibilité des données

Dans une table contenant de nombreux montants, l'affichage intégral de chaque valeur peut rapidement réduire la lisibilité.

Les valeurs importantes sont donc compactées dans les tableaux lorsque cela est pertinent, par exemple en millions.

Le détail de la transaction conserve cependant les valeurs complètes.

Cette approche permet de distinguer :

- une vue de scanning rapide ;

- une vue détaillée destinée à l’analyse d’une transaction.

7.3 Économie du trade

Les trois produits ne sont pas résumés de la même façon.

Exemples de représentations compactes :

- Bond : EUR 5.0m · Price 99.42;
- IRS : EUR 50.0m · Pay Fixed 2.85%;
- FX Forward : Buy EUR 10.0m / Sell USD 11.3m · Fwd 1.1300.

La représentation reste ainsi concise sans supprimer l’information économique essentielle.

7.4 Vue détaillée

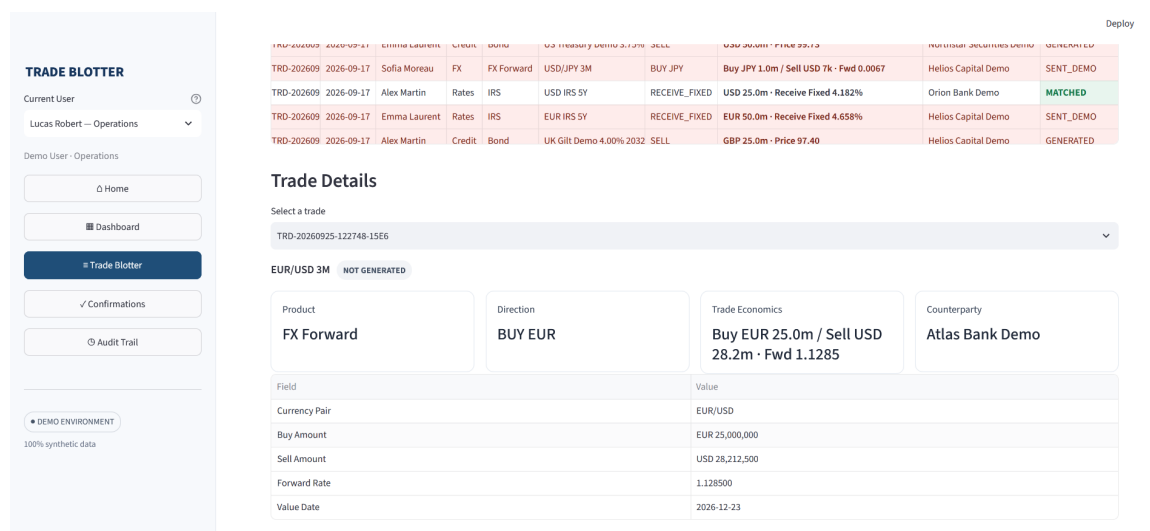


Figure 7.2 – Détail d’un FX Forward

La vue détaillée permet de revenir aux informations complètes lorsque l’utilisateur souhaite examiner une opération particulière.

Chapitre 8

Post-Trade Control Room

8.1 Objectif

Le dashboard n'a pas été conçu comme une simple collection de graphiques.

Son objectif principal est de répondre à plusieurs questions opérationnelles :

- combien de transactions sont présentes ?
- où se situent-elles dans le workflow ?
- quelles transactions nécessitent une attention ?
- depuis combien de temps sont-elles en attente ?
- quelles sont les positions nominales ?
- comment l'activité est-elle répartie ?

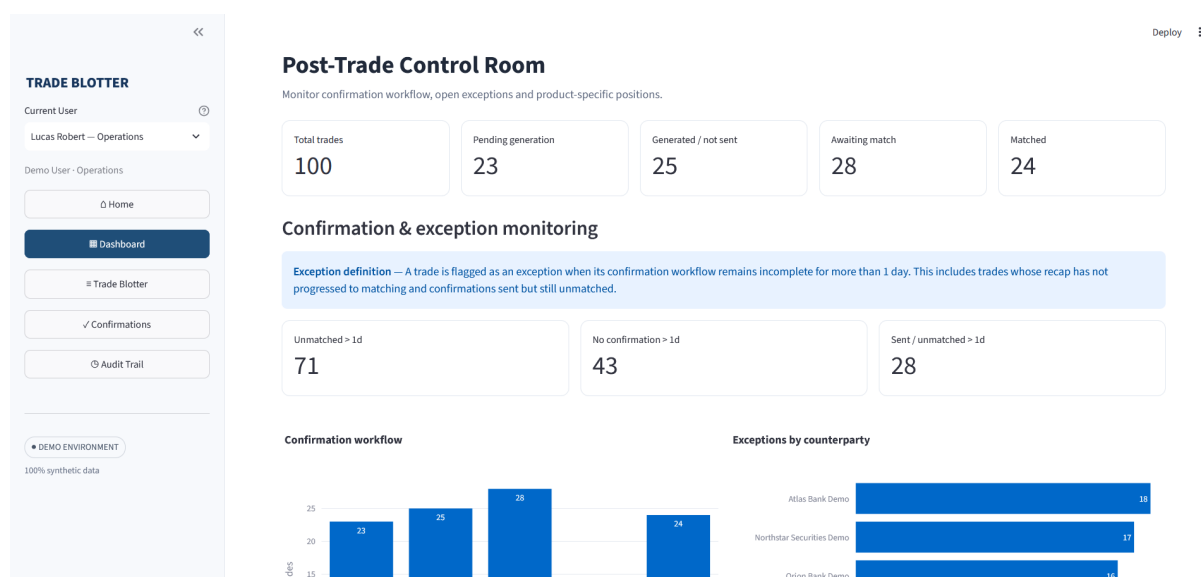


Figure 8.1 – Post-Trade Control Room

8.2 Indicateurs de workflow

Les indicateurs principaux donnent une lecture immédiate de la distribution des transactions :

- Pending Generation ;

- Generated / Not Sent ;
- Awaiting Match ;
- Matched.

Ces métriques transforment le workflow individuel de chaque transaction en vision agrégée du portefeuille.

8.3 Exception management

Toutes les transactions non finalisées ne nécessitent pas le même niveau d'attention.

Le projet utilise donc l'âge de la transaction et son statut afin de construire plusieurs indicateurs :

- Aged Unmatched > 1 day ;
- No Confirmation > 1 day ;
- Sent / Unmatched > 1 day.

L'objectif est de mettre en avant les opérations dont le workflow reste incomplet au-delà d'un seuil.

8.4 Oldest Open Exceptions

Un KPI indique combien d'exceptions existent, mais ne permet pas toujours de savoir lesquelles traiter en premier.

La table *Oldest Open Exceptions* complète donc les agrégats en présentant directement les transactions ouvertes les plus anciennes.

Cette vue rapproche le dashboard d'un outil opérationnel : l'utilisateur peut passer de la mesure globale à l'identification d'une transaction concrète.

8.5 Position Overview

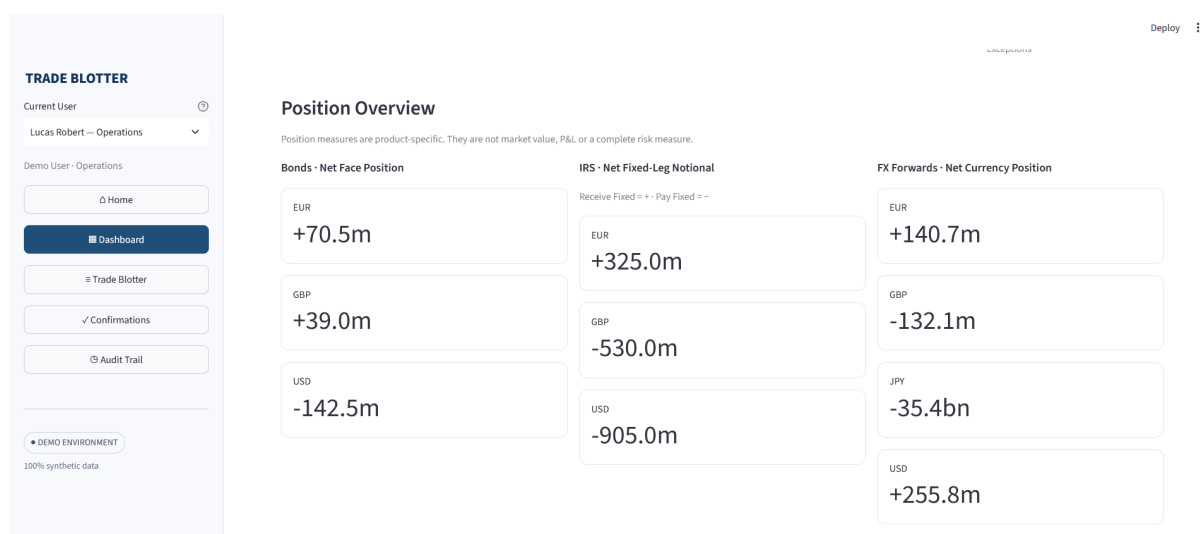


Figure 8.2 – Position Overview par type de produit

Les positions affichées sont adaptées à la nature du produit :

Produit	Mesure	Interprétation
Bond	Net Face Position	Nominal net BUY / SELL par devise
IRS	Net Fixed-Leg Notional	Receive Fixed positif, Pay Fixed négatif
FX Forward	Net Currency Position	Somme des jambes achetées et vendues par devise

Table 8.1 – Mesures utilisées dans Position Overview

8.6 Trading Analytics

La section Trading Analytics complète le contrôle opérationnel par des vues plus analytiques.

Elle permet notamment d'afficher :

- Trade Activity Over Time ;
- Trades by Trader ;
- Trades by Counterparty ;
- Confirmation Aging.

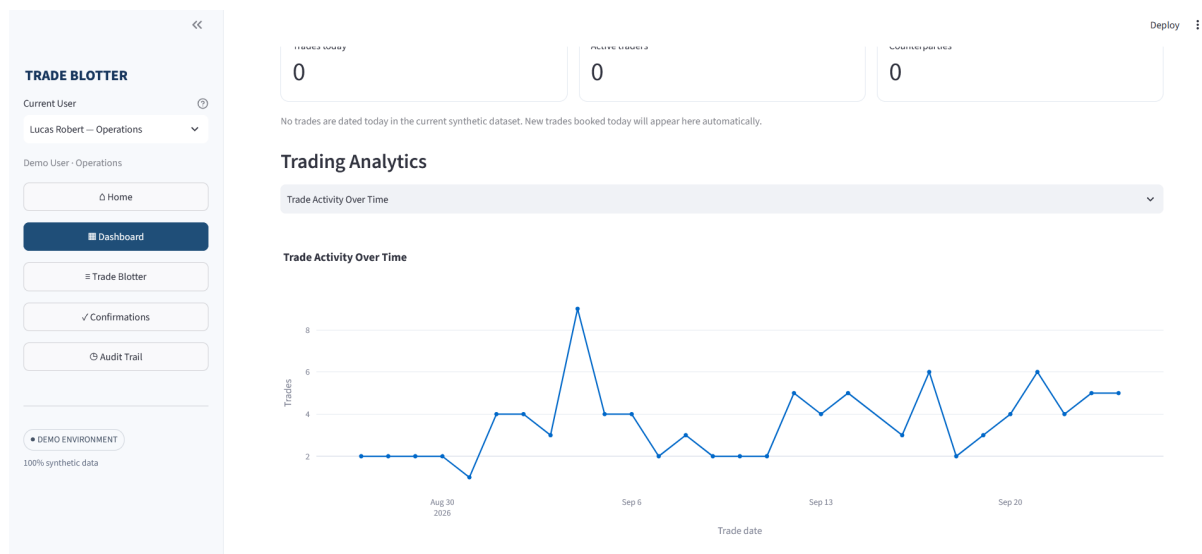


Figure 8.3 – Trade Activity Over Time

Le choix d'un sélecteur permettant d'afficher un graphique à la fois évite de surcharger l'écran tout en conservant plusieurs possibilités d'analyse.

Chapitre 9

Workflow de confirmation

9.1 De la transaction au Trade Recap

Lorsqu'une transaction est enregistrée dans le Trade Blotter, son traitement ne s'arrête pas à la création de l'enregistrement.

Elle entre dans un workflow représentant les différentes étapes de la confirmation.

Le statut initial signifie que la transaction existe dans la base, mais qu'aucun document n'a encore été généré.

9.2 Étape BOOKED

BOOKED représente le point de départ.

Les caractéristiques économiques sont présentes et peuvent être consultées, mais aucune confirmation n'a encore été produite.

À ce stade, l'action logique proposée à l'utilisateur Operations est donc la génération.

9.3 Étape GENERATED

L'action *Generate* produit un Trade Recap PDF à partir des données de la transaction.

Le statut devient alors **GENERATED**.

La génération n'est pas uniquement une modification visuelle : elle produit un artefact documentaire et un événement d'audit horodaté.

9.4 Étape SENT

L'étape suivante représente l'envoi de la confirmation.

Dans le projet, l'envoi est simulé.

Ce choix est volontaire : l'objectif est de démontrer la logique du workflow sans envoyer de véritables emails ni nécessiter de compte de messagerie ou de service externe.

Le timestamp de l'action est néanmoins enregistré afin de conserver une chronologie réaliste.

9.5 Étape MATCHED

Une transaction envoyée peut finalement être marquée comme **MATCHED**.

Cette étape représente la finalisation du workflow de démonstration.

Une transaction envoyée mais non matchée pendant une période prolongée reste au contraire visible dans les contrôles d'exception.

9.6 Interface dynamique

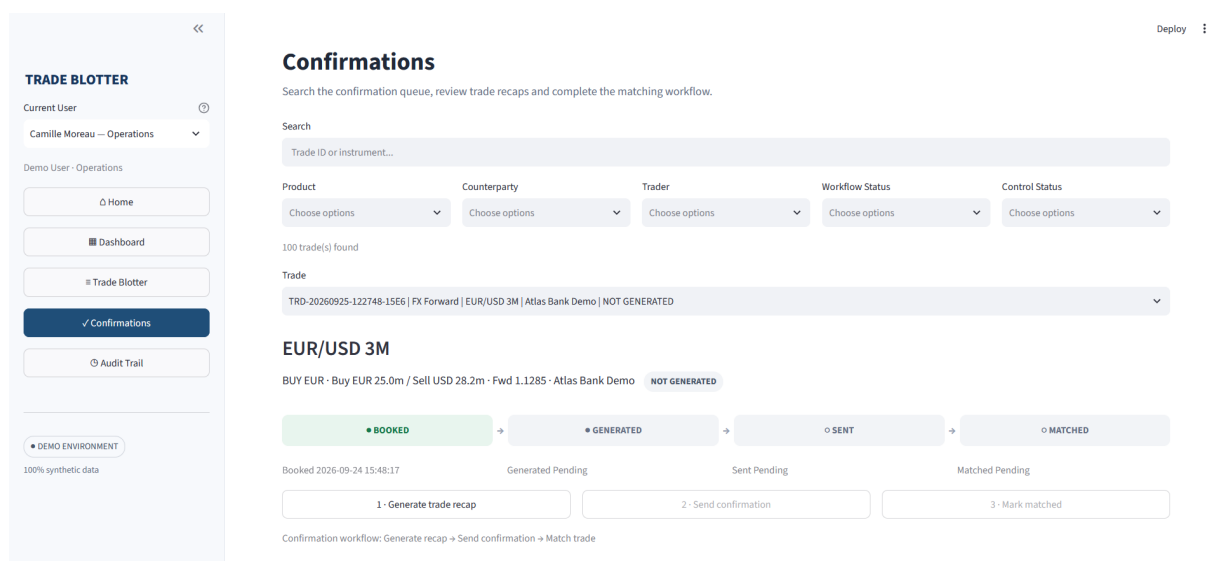


Figure 9.1 – *Interface de gestion des confirmations*

Les boutons disponibles dépendent du statut.

Cette logique empêche par exemple de proposer *Mark Matched* à une transaction dont la confirmation n'a jamais été générée.

Le workflow devient donc une véritable machine à états simplifiée plutôt qu'une simple valeur de texte.

Chapitre 10

Génération du Trade Recap PDF

10.1 Objectif

La confirmation générée doit fournir une représentation stable et lisible des informations de la transaction.

Le projet utilise ReportLab pour construire un document PDF au format institutionnel.

TRADE BLOTTER		TRADE RECAP
Post-Trade Management & Control Platform		FX Forward
DEMONSTRATION DOCUMENT · SYNTHETIC DATA · NOT A LEGAL CONFIRMATION		
TRADE REFERENCE		
Trade Reference	TRD-20260925-122748-15E6	
Trade Date / Time	2026-09-24 15:48:17	
Product	FX Forward	
Instrument	EUR/USD 3M	
TRADE ECONOMICS		
Currency Pair	EUR/USD	
Buy	EUR 25,000,000	
Sell	USD 28,212,500	
Forward Rate	1.128500	
Value Date	2026-12-23	
EXECUTION & COUNTERPARTY		
Trader / Desk	Lucas Bernard / FX	
Broker	Demo Broker Markets	
Counterparty	Atlas Bank Demo	
Counterparty LEI	529900DEMOATLAS001	
DOCUMENT INFORMATION		
Generated At	2026-09-25 12:42:07	
Delivery Channel	EMAIL	
Document Type	TRADE RECAP	
This recap is generated by a portfolio demonstration application. All entities, identifiers and trade data are synthetic and the document is not intended for live trading, settlement or legal reliance.		

Figure 10.1 – *Exemple de Trade Recap*

10.2 Organisation du document

Le document est organisé en plusieurs sections :

- Trade Reference ;
- Trade Economics ;
- Execution & Counterparty ;
- Document Information.

Les champs économiques sont adaptés au produit.

Un Bond n'affiche donc pas les mêmes informations qu'un IRS ou qu'un FX Forward.

10.3 Snapshot documentaire

Le Trade Recap représente un snapshot des informations disponibles lors de sa génération.

Le suivi du workflow continue ensuite dans l'application et dans l'audit.

Cette séparation permet de distinguer :

- le document produit à un instant donné ;
- l'état courant de la transaction dans le système.

10.4 Restauration du document

Le déploiement cloud peut conduire à la disparition d'un fichier généré alors que la base contient toujours un statut indiquant que la confirmation a déjà été produite.

Le projet prévoit donc la possibilité de reconstruire le PDF lorsqu'il manque, sans recréer artificiellement un second événement `CONFIRMATION_GENERATED`.

Cette distinction évite de confondre la restauration technique d'un fichier avec une nouvelle action métier.

Chapitre 11

Rôles et permissions

11.1 Pourquoi introduire des rôles ?

Dans une application opérationnelle, tous les utilisateurs n'ont pas les mêmes responsabilités.

Même si Trade Blotter n'implémente pas une authentification de production, le projet introduit plusieurs profils afin de représenter cette séparation.

11.2 Trader

Le Trader peut notamment créer de nouvelles transactions.

Il peut consulter les confirmations mais ne doit pas réaliser les actions opérationnelles de traitement.

11.3 Operations

Le profil Operations est orienté vers le traitement post-trade.

Il peut générer les confirmations, simuler leur envoi et effectuer le matching.

11.4 Supervisor

Le Supervisor dispose d'une vision plus large et peut notamment accéder à la piste d'audit.

11.5 Interface et backend

Masquer un bouton n'est pas une protection suffisante.

Le principe général est donc que les permissions importantes doivent également être respectées par la logique backend.

Cette approche reprend le même raisonnement que pour la validation : l'interface améliore l'expérience, mais la logique métier reste l'autorité.

Chapitre 12

Audit Trail

12.1 Objectif de la traçabilité

Connaître le statut actuel d'une transaction ne permet pas de comprendre comment elle est arrivée dans cet état.

L'Audit Trail conserve donc les événements significatifs du cycle de vie.

Un événement peut contenir :

- timestamp ;
- utilisateur ;
- rôle ;
- action ;
- Trade ID.

TRADE BLOTTER

Current User
Camille Moreau – Operations

Demo User - Operations

Home

Dashboard

Trade Blotter

Confirmations

Audit Trail

DEMO ENVIRONMENT
100% synthetic data

Audit Trail
Chronological record of booking, confirmation, delivery and matching events.

Action: Choose options

User: Choose options

Role: Choose options

Trade ID contains

Date range
2026 / 08 / 27 – 2026 / 09 / 25

254 events shown

Timestamp	User	Role	Action	Trade ID	Details
25/09/2026 12:42:07	Camille Moreau	Operations	Confirmation Generated	TRD-20260925-122748-15E6	TRD-20260925-122748-15E6_trade_recap.pdf
24/09/2026 15:48:17	Lucas Bernard	Trader	Trade Booked	TRD-20260925-122748-15E6	BUY 25,000,000 EUR / SELL USD
24/09/2026 15:03:25	Lucas Robert	Operations	Confirmation Generated	TRD-20260925-122757-829B	Synthetic seeded workflow event
24/09/2026 14:54:25	Sofia Moreau	Trader	Trade Booked	TRD-20260925-122757-829B	RECEIVE_FIXED 5,000,000 EUR EUR IRS 2Y
24/09/2026 11:41:25	Camille Moreau	Operations	Confirmation Generated	TRD-20260925-122752-0393	Synthetic seeded workflow event
24/09/2026 11:14:25	Emma Laurent	Trader	Trade Booked	TRD-20260925-122752-0393	RECEIVE_FIXED 100,000,000 EUR EUR IRS 2Y
24/09/2026 10:56:23	Lucas Robert	Operations	Confirmation Generated	TRD-20260925-122754-2F0C	Synthetic seeded workflow event
24/09/2026 10:33:23	Sofia Moreau	Trader	Trade Booked	TRD-20260925-122754-2F0C	BUY 2,000,000 JPY / SELL USD
24/09/2026 10:14:08	Alex Martin	Trader	Trade Booked	TRD-20260925-122747-44A6	RECEIVE_FIXED 5,000,000 USD USD IRS 5Y
23/09/2026 14:13:07	Thomas Leroy	Operations	Trade Matched	TRD-20260925-122756-328B	Synthetic seeded workflow event

Figure 12.1 – Piste d'audit de l'application

12.2 Source des timestamps

Les timestamps d'audit sont utilisés comme source de référence pour les dates de génération, d'envoi et de matching.

Pour les transactions historiques synthétiques, le seeder construit une chronologie cohérente :

$$t_{\text{BOOKED}} < t_{\text{GENERATED}} < t_{\text{SENT}} < t_{\text{MATCHED}}$$

Lorsqu'une action est effectuée réellement dans l'application, son événement est créé au moment de l'action.

Troisième partie

Réalisation technique

Chapitre 13

Services métier

13.1 Trade Service

Le Trade Service centralise les opérations directement liées aux transactions.

Il est notamment responsable de la validation, du booking et de la récupération des trades.

Centraliser ces règles évite qu'une validation importante existe uniquement dans le formulaire Streamlit.

13.2 Confirmation Service

Le Confirmation Service gère les éléments du workflow de confirmation et la production du Trade Recap.

Cette séparation permet de ne pas mélanger le rendu de l'interface avec la logique documentaire.

13.3 Audit Service

L'Audit Service centralise la création et la récupération des événements d'audit.

Il permet aux différents composants d'utiliser une méthode cohérente pour enregistrer les actions.

13.4 Counterparty Service

Le Counterparty Service fournit les contreparties synthétiques utilisées dans les formulaires et les transactions.

13.5 User Service

Le User Service fournit les utilisateurs et leurs rôles de démonstration.

13.6 Position Service

Le service de position agrège les transactions selon une logique spécifique au produit.

Il ne cherche volontairement pas à fournir une mesure générique appelée “exposure”, car les calculs réalisés n’ont pas tous la même signification financière.

13.7 Email Service

Le service email représente l’étape d’envoi sans transmettre réellement de message.

Cette abstraction permet de conserver un workflow crédible tout en évitant les risques et la configuration associés à un service de messagerie réel.

Chapitre 14

Dataset synthétique

14.1 Pourquoi générer des données ?

Une application de contrôle est difficile à démontrer avec une base vide.

Un jeu de données suffisamment varié est nécessaire pour faire apparaître :

- plusieurs produits ;
- plusieurs devises ;
- différents traders ;
- différentes contreparties ;
- plusieurs états de confirmation ;
- des exceptions anciennes ;
- des transactions matchées.

Le script `seed_demo.py` génère donc un environnement initial de 100 transactions synthétiques.

14.2 Cohérence du dataset

Le dataset ne doit pas seulement contenir des valeurs aléatoires.

Les dates et les statuts doivent rester cohérents.

Une transaction matchée doit par exemple disposer d'un historique permettant de retrouver les étapes précédentes.

Cette cohérence est particulièrement importante car le dashboard et l'audit reposent sur ces informations.

14.3 Données obligataires

Des descriptions synthétiques sont utilisées, par exemple :

- OAT Demo 3.00% 2031 ;
- Bund Demo 2.60% 2033 ;
- UK Gilt Demo 4.00% 2032 ;

— US Treasury Demo 3.75% 2030.

Le terme *Demo* permet de rappeler immédiatement qu'il ne s'agit pas d'un référentiel réel.

Chapitre 15

Tests et qualité

15.1 Pourquoi tester le backend ?

La validation côté interface peut évoluer.

Un changement de widget ne doit pas permettre à une transaction invalide d'entrer dans la base.

Les tests backend servent donc à vérifier les invariants métier indépendamment de Streamlit.

15.2 Scénarios testés

Les tests couvrent différents scénarios de validation, notamment autour :

- des montants ;
- des dates ;
- des caractéristiques produit ;
- de la cohérence des devises ;
- des contraintes structurelles.

15.3 Principe d'intégrité

L'un des invariants les plus importants peut être résumé ainsi :

Une transaction invalide ne doit produire aucun effet métier persistant.

Cela signifie qu'elle ne doit pas apparaître dans la base, recevoir de Trade ID ou produire un événement de booking.

Chapitre 16

Interface et expérience utilisateur

16.1 Direction visuelle

L'interface utilise principalement une palette bleu marine, bleu clair, blanc et gris.

Les couleurs rouge, orange et verte sont réservées autant que possible aux états nécessitant une signification particulière.

Cette approche vise à produire une interface sobre et orientée outil professionnel plutôt qu'une démonstration très décorative.

16.2 Navigation

La navigation latérale a été personnalisée afin d'obtenir des boutons plus lisibles qu'un simple groupe de boutons radio.

La page active reste visuellement identifiable et le sélecteur d'utilisateur de démonstration est conservé dans la sidebar.

16.3 Page d'accueil

La page Home présente immédiatement :

- le nom du projet ;
- sa fonction ;
- les principales capacités ;
- les technologies ;
- l'auteur ;
- le caractère synthétique des données.

L'objectif est qu'un utilisateur découvrant la démo comprenne son contexte avant de naviguer dans les fonctionnalités.

Quatrième partie

Du développement au déploiement

Chapitre 17

Versionnement avec Git

17.1 Objectif

Git est utilisé pour conserver l'historique du code et construire une version publiable du projet. Le dépôt distant est hébergé sur GitHub.

17.2 Fichiers exclus

Certains éléments ne doivent pas être versionnés.

Le fichier `.gitignore` exclut notamment :

- `.venv/`;
- `.env`;
- les fichiers `__pycache__`;
- les bases `data/*.db`;
- les PDF générés;
- les caches Pytest.

Cette distinction permet de publier le code sans embarquer l'environnement Python local, les données de travail ou d'éventuels secrets.

17.3 Dépôt public

Le code source est disponible à l'adresse :

<https://github.com/GasparRaybaudMabily/trade-blotter>

Le README sert de point d'entrée pour présenter le projet, expliquer son installation et orienter vers la démonstration publique.

Chapitre 18

Déploiement sur Streamlit Community Cloud

18.1 Objectif du déploiement

Une application de portfolio est plus facilement évaluée lorsqu'elle peut être utilisée directement.

Le déploiement public évite à un visiteur de devoir cloner le dépôt, installer Python et lancer manuellement Streamlit.

La démonstration est disponible à l'adresse :

<https://trade-blotter-gaspar-raybaud-mabily.streamlit.app>

18.2 Problème de la base vide

La base SQLite locale est volontairement exclue du dépôt Git.

Cette décision est correcte pour la propreté du repository, mais elle a une conséquence : une nouvelle instance déployée depuis GitHub peut démarrer sans transactions.

Lors du premier déploiement, l'application pouvait donc fonctionner techniquement tout en affichant un dashboard vide.

18.3 Solution retenue

Le démarrage de l'application vérifie si des transactions existent.

De manière simplifiée :

```
1 init_db()
2
3 if not list_trades():
4     seed()
```

Si la base est vide, le dataset synthétique est créé automatiquement.

Dans le cas contraire, aucune réinitialisation n'est effectuée.

18.4 Pourquoi ne pas appeler seed à chaque démarrage ?

Le seeder reconstruit le dataset.

L'appeler systématiquement à chaque rerun Streamlit aurait donc supprimé les actions réalisées par l'utilisateur.

Un trade créé dans l'application aurait disparu dès le prochain rerun.

La condition `if not list_trades()` résout ce problème :

- une instance vide devient immédiatement démontrable ;
- une instance déjà utilisée conserve ses données.

18.5 Persistance et limite du cloud

La persistance du stockage local d'une application cloud ne doit cependant pas être assimilée à celle d'une base de production durable.

Les données peuvent survivre à des sessions et reruns, mais l'application est conçue de façon à pouvoir reconstruire son dataset synthétique si une nouvelle instance démarre avec une base vide.

Ce comportement est cohérent avec la nature de la plateforme : une démonstration et non un système d'enregistrement officiel.

Cinquième partie

Analyse critique et perspectives

Chapitre 19

Résultats obtenus

19.1 Résultat fonctionnel

Le projet aboutit à une application permettant de parcourir un cycle post-trade complet de démonstration.

Un utilisateur peut :

1. saisir une transaction ;
2. déclencher sa validation ;
3. la retrouver dans le blotter ;
4. consulter son économie ;
5. générer son Trade Recap ;
6. simuler son envoi ;
7. la matcher ;
8. retrouver les événements dans l'audit ;
9. observer l'impact du statut dans le dashboard.

L'intérêt principal est donc l'intégration entre les fonctionnalités plutôt que l'existence isolée de chacune d'entre elles.

19.2 Résultat technique

Le projet combine plusieurs problématiques :

- interface interactive ;
- modélisation relationnelle ;
- règles métier ;
- workflow ;
- génération PDF ;
- visualisation de données ;
- tests ;
- versionnement ;
- déploiement.

Cette diversité a nécessité de maintenir une cohérence entre les différentes couches du système.

Chapitre 20

Limites

20.1 Authentification

Le sélecteur Current User représente des profils de démonstration.

Il ne constitue pas un véritable mécanisme d'authentification.

Une application réelle nécessiterait une gestion sécurisée des identités, des sessions et des permissions.

20.2 Base de données

SQLite est adapté au projet mais ne représente pas nécessairement le choix d'une application institutionnelle multi-utilisateurs.

Une évolution vers PostgreSQL ou un autre SGBD serveur permettrait une gestion plus robuste de la concurrence et de l'exploitation.

20.3 Confirmations

L'envoi est simulé.

Aucun email ou message institutionnel réel n'est transmis.

Une version plus avancée pourrait intégrer un service de messagerie, une API ou une plateforme externe.

20.4 Risk Management

Le projet ne calcule pas de valorisation complète.

Il ne fournit notamment pas :

- Mark-to-Market ;
- P&L ;
- DV01 / PV01 ;
- duration ;

- sensibilités aux courbes ;
- scénarios de marché ;
- VaR.

Les vues de positions doivent donc être interprétées dans leur périmètre nominal.

20.5 Market data

Aucune source de données de marché temps réel n'est utilisée.

Les caractéristiques du dataset sont synthétiques.

Cette décision permet d'assurer que la démonstration ne dépend d'aucun flux externe et ne présente aucune donnée réelle comme si elle provenait d'un marché.

Chapitre 21

Perspectives d'évolution

21.1 Module Risk Management

Une évolution naturelle serait d'ajouter un véritable module de risque.

Pour les obligations, il pourrait intégrer :

- dirty price ;
- accrued interest ;
- market value ;
- modified duration ;
- DV01.

Pour les IRS :

- actualisation des cash-flows ;
- courbes de taux ;
- NPV ;
- DV01 / PV01.

Pour les FX Forwards :

- spot market data ;
- forward points ;
- MtM ;
- sensibilités par devise.

21.2 API et séparation frontend/backend

Une version plus proche d'une architecture de production pourrait séparer :

- un backend Python exposant une API ;
- un frontend dédié ;
- une base PostgreSQL ;
- un service d'authentification.

Cette évolution améliorerait notamment la capacité multi-utilisateurs et la scalabilité.

21.3 Matching automatisé

Le matching est actuellement déclenché manuellement.

Une évolution pourrait comparer automatiquement deux représentations d'une transaction et détecter des breaks sur :

- montants ;
- devises ;
- dates ;
- taux ;
- sens ;
- contrepartie.

Le dashboard pourrait ensuite distinguer les transactions matched des transactions présentant un véritable mismatch économique.

21.4 Gestion documentaire

Le Trade Recap pourrait également évoluer vers un module documentaire plus complet :

- versionnement des documents ;
- archivage ;
- templates par produit ;
- historique des téléchargements ;
- envoi automatisé.

Chapitre 22

Conclusion

Le développement de Trade Blotter a permis de construire une application complète autour d'un cas d'usage post-trade.

Le projet a commencé par la représentation de transactions financières, mais a progressivement nécessité de traiter des problématiques plus larges : architecture, validation, persistance, workflow, contrôle, audit, visualisation, génération documentaire, tests et déploiement.

La prise en charge de trois produits différents a également montré l'importance d'éviter une modélisation excessivement générique. Les Bonds, IRS et FX Forwards partagent certaines caractéristiques, mais leur économie et leurs positions doivent conserver une représentation adaptée.

La validation backend constitue un autre élément central. Les restrictions visuelles de l'interface sont utiles pour guider l'utilisateur, mais l'intégrité des données doit être garantie indépendamment de la présentation.

Le workflow de confirmation relie ensuite la transaction à son traitement opérationnel. Les étapes **BOOKED**, **GENERATED**, **SENT** et **MATCHED** alimentent à la fois l'interface, les contrôles et la piste d'audit.

Le Post-Trade Control Room transforme ces données individuelles en vision agrégée. Il permet de suivre le workflow, d'identifier les exceptions, d'analyser l'activité et de visualiser les positions nominales.

Enfin, la publication sur GitHub et le déploiement sur Streamlit Community Cloud ont permis de transformer le projet local en démonstration accessible. Cette dernière étape a également introduit des problématiques concrètes, comme l'initialisation d'une base vide sans réinitialiser les données à chaque rerun.

Trade Blotter reste volontairement une application de démonstration. Il ne cherche pas à remplacer une infrastructure financière de production. Cependant, son architecture permet d'identifier clairement plusieurs pistes d'évolution : véritable authentification, base serveur, matching automatisé, market data, valorisation et Risk Management.

Le projet constitue ainsi une mise en pratique de concepts financiers, opérationnels et logiciels autour d'un même système cohérent.

TRADE BLOTTER

Post-Trade Management & Control Platform

Gaspar RAYBAUD-MABILY

2026

<https://github.com/GasparRaybaudMabilly/trade-blotter>

<https://trade-blotter-gaspar-raybaud-mabilly.streamlit.app>